instituto superior de engenharia de lisboa

LMATE

Programação

Inverno 2026-2027

Trabalho 6

1

Exercício 3 - Outro preço

- O preço de um produto é 135 euros e está com um desconto de 25%.
- Escreva um programa que calcule e mostre (faça *output*), o fator de desconto (a divisão da percentagem de desconto por 100), o valor do desconto e o preço a pagar pelo produto.

Solução

- Usar o programa do exercício 2 (que já faz o que é pedido), e alterar o preço do primeiro produto produto (e apagar o segundo).
- Pretende-se manter inalterada a solução do exercício 2, por isso copiou-se o ficheiro `exercicio_2_precos.py` para `exercicio_3_precos_alterar.py`.

Programa (no ficheiro `exercicio_3_precos_alterar.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
# copiado do ficheiro exercicio_2_precos.py e comentado porque vai ser adaptado
# print('preço do produto 1:            ', 40, 'em euros')
# print('desconto do produto 1:         ', 25, 'porcento')
# print('fator de desconto do produto 1:', 25/100)
# print('desconto do produto 1:         ', 40*25/100, 'euros')
# print('preço a pagar pelo produto 1:  ', 40 - 40*25/100, 'euros')

# copiado de cima e adaptado
print('preço do produto:            ', 135, 'em euros')
print('desconto do produto:         ', 25, 'porcento')
print('fator de desconto do produto:', 25/100)
print('desconto do produto:         ', 135*25/100, 'euros')
print('preço a pagar pelo produto:  ', 135 - 135*25/100, 'euros')
```

Output

```
preço do produto:             135 em euros
desconto do produto:          25 porcento
fator de desconto do produto: 0.25
desconto do produto:          33.75 euros
preço a pagar pelo produto:   101.25 euros
```

Inaceitável!!!

- Porque é que um programador considera esta solução inaceitável?
- Porque o produto é só um. Só se deveria ter que alterar um preço. Mas o valor do preço aparece 4 vezes no programa `exercicio_2_precos.py`. Por isso apesar do preço a alterar ser só um, o programa tem que ser alterado em 4 sítios diferentes. E isso é inaceitável.

Solução - usar variáveis

Programa (no ficheiro `exercicio_3_precos_alterar_v2.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
preço = 135
print('preço do produto:            ', preço, 'em euros')
print('desconto do produto:         ', 25, 'porcento')
print('fator de desconto do produto:', 25/100)
print('desconto do produto:         ', preço*25/100, 'euros')
print('preço a pagar pelo produto:  ', preço - preço*25/100, 'euros')
```

Output

```
preço do produto:             135 em euros
desconto do produto:          25 porcento
fator de desconto do produto: 0.25
desconto do produto:          33.75 euros
preço a pagar pelo produto:   101.25 euros
```

Variáveis

- As variáveis permitem armazenar valores na memória dos computadores.
- Uma variável tem um nome, e é inicializada com um valor. Usando o operador de atribuição, o sinal de igual `=`.
- Do lado esquerdo do sinal de igual tem que estar o nome de uma variável. Do lado direito têm que estar um valor. (Ou código Python cuja execução resulte num valor. Por exemplo `2*3/4`.) A variável do lado esquerdo passa a armazenar o valor do lado direito. Diz-se que o valor do lado direito é o valor da variável.
- Quando a variável é usada, é substituída pelo seu valor, isto é, o valor que armazena.
- **IMPORTANTE!** Uma variável **só pode ser usada depois de ter sido inicializada**.

Exemplos (no ficheiro `exemplos_variaveis.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
mensagem = 'Hello Variables!'
print(mensagem) # a variável mensagem é substituída pelo seu valor
x = 10
y = x + 1 # a variável x é substitída por 10, e é executada a
          # instrução 10 + 1 cujo resultado é 11. O valor 11 é
          # armazenado na variável y
x = 30
print(x)
print(y)
# a próxima linha dá erro porque a variável z não foi inicializada, não está definida
print(z)
```

Output

```
Hello Variables!
30
11
Traceback (most recent call last):
  File "/home/jbs/develop/202309_hello_python_world_v2/pt/variaveis/exemplos_variaveis.py", line 11, in <mod
    print(z)
          ^
NameError: name 'z' is not defined
```

Nomes de variáveis

- Os nomes das variáveis não podem ser quaisquer. Obdecem a algumas regras, nomeadamente:
  - Só podem ter letras, algarismos e o sublinhado _ (*underscore*, em inglês). Não podem, por exemplo, ter espaços. Nem outros símbolos, como os dos operadores aritméticos, entre outros.
  - Não podem começar por um algarismo. Apenas por uma letra ou por um *underscore*.
- Há outros aspetos a ter em consideração relativamente aos nomes das variáveis, nomeadamente:
  - Os nomes em Pyhton, em particular os nomes das variáveis, são *case sensitive*, o que quer dizer que as letras minúsculas são consideradas diferentes das maiúsculas correspondentes. Por exemplo, os nomes `mensagem` e `Mensagem`, são considerados nomes de variáveis diferentes. E isto aplica-se a todos os nomes Pyhton. Por exemplo, aos nomes das funções. A função `print` existe. Mas a função `Print` não existe.
  - Por convenção os nomes de variáveis que sejam constituidos por várias palavras devem ser escritos com as palavras em minúsculas, separadas por *underscores*. Por exemplo, `mensagem_inicial`.
  - Os nomes das variáveis devem ser sugestivos.

Pergunta

- O programa que se segue é válido? Qual é o seu *output*?

Programa (no ficheiro `pergunta_1_variaveis.py`)

```
x = 5
print(x)
x = x + 1
print(x)
```

Resposta

- Sim, o programa é válido. Quando é executado `x + 1`, a variável `x` é substituída pelo seu valor, isto é, `5`. Como `5 + 1 = 6`, o valor do lado direito do sinal de igual é `6`. Esse valor é armazenado na variável cujo nome está do lado esquerdo do sinal de igual. Que é `x`. Portanto o valor de `x` passa a ser `6`.
- Este tipo de construção é muito útil para incrementar o valor de uma variável.

O *output* deste programa é:

Pergunta

- O programa que se segue é válido? Qual é o seu *output*?

Programa (no ficheiro `pergunta_2_variaveis.py`)

```
x = x + 1
print(x)
```

Resposta

- Não, o programa não é válido. Quando é executado `x + 1`, a variável `x` não está definida. Dá erro. O *output* deste programa é um erro porque na linha 1 a variável `x` não está definida.

Operadores de atribuição

- As atribuições do género `x = x + 1` são tão usadas, que existe um operador de atribuição equivalente e mais conciso. É `x += 1`. Incrementa à variável `x` o valor `1`.
- Existem também operadores de atribuição com incremento para as restantes operações aritméticas.

Exemplo (no ficheiro `exemplos_operadores_atribuicao`)

```
x = 0
print(x)

x += 1 # é equivalente a x = x + 1. este operador += é muito usado
print(x)

x -= 2 # é equivalente a x = x - 2
print(x)

x *= 3 # é equivalente a x = x * 3
print(x)

x /= 4 # é equivalente a x = x / 4
print(x)

x **= 5 # é equivalente a x = x ** 5
print(x)

x %= 6 # é equivalente a x = x % 6
print(x)

x //= 7 # é equivalente a x = x // 7
print(x)
```

Output

```
0
1
-1
-3
-0.75
-0.2373046875
5.7626953125
0.0
```

Para fazer

Considere o programa Python 3, que se segue.

```
x = 1
print(x)
y = 2
print(y)
z = x + y
print(z)
x = x + 1
print(x)
y = y + 1
print(y)
z = z + 1
print(z)
```

Confirme que o *output* do programa é o que se segue.

```
1
2
3
2
3
4
```

Indique se as afirmações que se seguem são verdadeiras ou falsas.

1.1

A execução de

```
i = 5
print(I)
```

dá erro.

1.2

A execução de

```
print(q)
q = 2
```

não dá erro.

1.3

A execução de

```
2f = 4
```

não dá erro.

1.4

A execução de

```
j = 1
e = 2
m n = j + e
```

não dá erro.

1.5

A execução de

```
v = 1
t = v + 1
v = v + 1
r = r + 1
```

dá erro.